Layer-wise Statistical Analysis: A Time-Sensitive Framework for Implementing Statistical Methods in Neural Network Training Sessions

Roman Budjač[1] and Maroš Valášek[2]

[1*]Research centre, University of Žilina, Univerzitná 8215/1, Žilina, 010 26, Slovakia.

[2]Faculty of Electrical Engineering and Information Technology, University of Žilina, Univerzitná 8215/1, Žilina, 010 26, Slovakia.

Contributing authors: roman.budjac@uniza.sk; maros.valasek@feit.uniza.sk;

Abstract

Interrupting neural network training is a fundamental operation that facilitates critical interventions, such as early stopping and dynamic layer-wise analysis, during the training process. Early stopping utilises interruptions to prevent overfitting by terminating training when the performance metrics on a validation set plateau or deteriorate. Similarly, layer-wise analysis during interruptions provides insights into the internal behaviour of the network, including weight distributions, gradient flow, and activation patterns, which are essential for diagnosing training inefficiencies or identifying opportunities for optimisation. This article proposed a new framework for implementing statistical methods in training sessions. Moreover, in this paper, the time analysis of opened interruption windows and conducted data for statistical analysis, particularly the Kulback-Leibner divergence.

Keywords: deep learning, neural networks, training interruption, layer-wise analysis, early stopping, model optimisation, quantisation

1 Introduction

The field of deep learning has seen remarkable advancements in recent years, with neural networks demonstrating impressive performances across a wide range of tasks. However, the computational complexity of training these models often presents a significant challenge, particularly as the depth and width of the networks continue to increase [1]. Quantisation in neural networks reduces the precision of the network's parameters and computations, typically from floating-point to fixed-point or integer representations [2, 3]. This technique is motivated by several factors, including reducing model size, decreasing memory requirements, and enhancing computational efficiency, particularly on resource-constrained devices [4, 5]. By converting high-precision values to lower-precision formats, quantisation can significantly compress neural networks without a substantial loss in accuracy [6]. The process involves mapping continuous or high-precision discrete values to a smaller set of values, often utilising techniques such as rounding or binning [7]. This approach is particularly advantageous for deploying deep learning models on edge devices, mobile platforms, and IoT devices, where power consumption and processing capabilities are limited [8]. Additionally, quantisation can lead to faster inference times and reduced energy consumption, rendering it an essential tool for optimising neural networks for real-world applications [4, 5]. This demonstrates that quantisation is an essential component in training neural network models. Consequently, quantisation plays an indispensable role in the field of artificial intelligence, particularly in instances where the dataset does not meet industrial standards for the problem being addressed and consists of real-world data specific to a given situation [9]. By implementing quantisation techniques, researchers and practitioners can significantly enhance the efficiency of neural network models. This optimisation leads to a reduced memory footprint, decreased power consumption, and improved inference speed. These benefits are particularly crucial when deploying models on resource-constrained devices, such as mobile phones, embedded systems, or edge computing platforms.

In this research, while quantisation significantly reduces the model size and computational requirements by compressing weights into lower-precision formats, understanding its precise impact on model behaviour requires careful analysis throughout the training process. This creates an internal tension with the need to interrupt training to measure quantisation effects, but these interruptions can potentially disrupt the subtle dynamics of optimisation. The challenge of analysing the impact of quantisation without compromising the integrity of the training becomes particularly important when trying to identify optimal points for refinement reduction or when studying how different layers respond to different quantisation schemes. This paper proposes an innovative method for interrupting neural network training processes through an intelligent checkpoint mechanism that enables layer-wise data analysis based on KL divergence. Our approach allows for a detailed statistical examination of network layers during training, providing valuable insights into layer behaviour and transformation characteristics. In future work, we aim to integrate this framework into quantisation-aware training processes to gain a better understanding of how quantised layers evolve compared to their non-quantised counterparts. This advancement

will facilitate more informed decisions during model compression, potentially leading to improved quantisation strategies that preserve model accuracy while reducing computational requirements.

1.1 Interrupting training process for layer-wise statistical analysis

Interrupting neural network training to analyse individual layers presents several significant challenges. First, interrupting the training process can derail the model's learning path, potentially causing it to lose previously learned information or fail to reach its optimal state. There is also complexity in measuring layer differences using KL divergence, as it requires choosing an appropriate baseline distribution - a task that becomes hard when the network constantly changes. The process is further complicated by the substantial computational cost of performing these layer-wise measurements repeatedly during training. To address these challenges, researchers need to develop better methods, such as smarter checkpoint timing or alternative measurement techniques that do not require completely stopping the training process.

2 Materials and methods

This paper proposes an innovative framework based on an intelligent checkpoint that allows the training of the neural network to be interrupted in order to perform statistical analysis during the process. The research addresses the broader problem of model quantisation, where we aim to include statistical analysis of the layers during the training phase rather than afterwards. This approach allows making more informed quantisation decisions by leveraging real-time statistical data during training. Implementing quantisation during neural network training can also be combined with other techniques, such as pruning, to optimise the network's performance further. To gain insight into the training process's internal behaviour, examining how weights are distributed throughout the network is valuable.

One way to analyse the training process is to compare the distribution of weights in each neural network layer across different training iterations. Specifically, the Kullback-Leibler divergence is a widely used metric for quantifying the difference between two probability distributions, and it can be employed to compare the weight distributions in successive training steps, providing insights into the evolution of the network's internal representations. By monitoring the KL divergence between the weight distributions in each layer, researchers and practitioners can gain a deeper understanding of how the network is learning and identify potential issues or bottlenecks in the training process. This information can then be used to devise more effective training strategies, such as interrupting the training process and applying statistical methods to adjust the hyperparameters or architectural choices [10]. Tracking the KL divergence between weight distributions in each layer can help researchers and practitioners gain a deeper understanding of how the network is learning, enabling the identification of potential issues or bottlenecks in the training process.

By monitoring the weight distributions across layers with statistical methods, researchers can identify which layers are undergoing the most significant changes

during training and potentially utilise this information to guide the application of techniques such as quantisation or pruning to optimise the network performance. This approach enables the assessment of weight distribution quality during training and facilitates inferences regarding the efficacy of the training process. Moreover, the magnitude of the statistical difference between distributions can serve as a criterion for interrupting training and implementing quantisation or other optimisation techniques. In conclusion, interrupting neural network training to analyse weight distribution can provide valuable insights into the training process and enable more efficient optimisation techniques [11].

Figure 1 depicts the general concept behind interrupting neural networks based on triggering checkpoints. After the first steps, such as loading the data set, The second step is initialising the model. This part contains setting hyperparameters for model training and other unnecessary processes for training.

2.1 Handling Training Interruptions in Quantised Neural Networks

Implementing interruption handling in neural network training necessitates careful consideration of three fundamental component groups: state preservation, checkpointing infrastructure, and recovery mechanisms. Implementing an interrupt algorithm during neural network training requires carefully considering three basic components: state preservation, checkpoint infrastructure, and recovery mechanisms.

Preserving the knowledge gained during the training of the model involves capturing and storing the current state of the neural network, including model parameters, optimisation states, and any other relevant training information.

State Preservation Components constitute the foundation of any resumable training system.

When restoring again, the system restores all saved states from the checkpoint, including model parameters and training progress markers. The training position must be accurately identified to avoid duplication or omission of data.

Data uploaders require proper repositioning to continue from the correct batch, while verification checks confirm that the behaviour of the restored model matches the performance before the interrupt. This is ensured by the implementation in the Pytorch/Keras library.

The integration of these components creates a resilient training system capable of handling interruptions while maintaining quantisation benefits. This approach minimises computational waste and ensures training continuity, which is particularly crucial for resource-intensive deep learning projects. The overhead of implementing such a system is negligible compared to the potential cost of training restarts, making it an essential element of production-grade deep learning implementations.

Best practices for system implementation include comprehensive logging of all saved states, automated validation of restored models, and graceful handling of edge cases such as corrupted checkpoints or parameter mismatches. Regular testing of the restoration process ensures reliability when interruptions occur in production environments.

This systematic approach to handling training interruptions provides the foundation for reliable deep learning model development, particularly when working with quantised networks where precision and state management become increasingly critical.

In the Fig X there is the implementation of techniques using interruption neural network training process for interrupting neural network process for quantising purposes. This is the basic mechanism of interrupting neural networks during training, incorporating the checkpointing mechanism, providing statistical analysis and continuing the process.

There are several techniques to analyse weight distributions, such as the KL divergence, Wasserstein distance, Kolmogorov-Smirnov test, and other statistical methods [12, 13, 14]. This is the place where analysis can be provided. For this purpose, we describe this place as an interruption window where, after interrupting the training process, we can use this state to provide some sort of calculation. In this particular case, statistical analysis is used.

2.1.1 Interruption time window

Interruption time windows are periodic checkpoints during training where you temporarily pause to evaluate the model's performance and decide whether to continue training or stop early. In the realm of deep learning model training, interruption time windows serve as key components for assessment and refinement. These strategically implemented pauses act as critical milestones throughout the developmental process, allowing researchers and data scientists to scrutinise progress and formulate well-informed decisions regarding the model's trajectory. By incorporating these intervals, the model can be effectively evaluated and optimised their models, ensuring a more robust and efficient training lifecycle [15].

2.1.2 Negative impact of interrupting the training process

The negative characteristics of the transfer of calculations in the interrupt com window during the training of neural networks are based on the nature of the mechanism of interrupting the training by the power of checkpoints. In this case, the obvious negative is the prolongation of the training time.

Implementing discontinuities in the layers during neural network training for statistical analysis poses significant challenges. In this case, a significant negative manifestation is the prolongation of the training period. The main concern lies in the disruption in the optimisation dynamics, whereby training interruptions can cause suboptimal convergence patterns or induce undesirable forms reminiscent of memory loss effects. Moreover, using the Kullback-Leibler divergence as an analytical metric requires establishing a reference distribution. This is considered a non-trivial task due to the dynamic nature of the network parameters during training. This complexity is compounded by the considerable computational burden associated with layer-wise divergence calculations across multiple temporal checkpoints, potentially compromising training efficiency. These challenge the development of sophisticated mitigation

strategies such as adaptive checkpointing mechanisms. The second path is implementing surrogate metrics that approximate KL divergence while maintaining training continuity.

Negative characteristics of interrupting neural network training can be categorised as follows:

- Losing time momentum occurs when training pauses disrupt the natural flow of optimisation.
- Larger models experience more significant overhead due to the sheer volume of parameters that must be saved, analysed, and reloaded. The checkpoint size scales linearly with model parameter counts.
- Complex architectures require more extensive analysis. Resuming training correctly becomes exponentially more difficult as architectural complexity increases, requiring extensive verification that all components have properly restored their states.
- Scale-dependent performance implications manifest non-linearly as models grow.

2.2 Layer-wise statistical analysis framework

The concept illustrated in Figure 1 demonstrates the process of interrupting a neural network based on a triggering checkpoint. This approach begins with essential preliminary steps, including loading the dataset and initialising the model. The initialisation phase encompasses setting crucial hyperparameters that govern the model's training process, such as learning rate, batch size, and number of epochs. Additionally, this stage may involve other preparatory tasks that, while not directly related to the training process itself, are necessary for the system's overall functionality.

Following these initial steps, the neural network training commences. However, the key feature of this approach is implementing a checkpoint mechanism. This mechanism allows for periodic interruptions in the training process, typically at predefined intervals or when specific conditions are met. During these interruptions, the current state of the model, including its weights and biases, can be saved. This checkpoint system serves multiple purposes, such as enabling the resumption of training from a saved state in case of unexpected terminations, facilitating model evaluation at different stages of training, and allowing for the implementation of early stopping techniques to prevent overfitting.

3 Results

As the research problems moved forward, we implemented the proposed framework for data analysis using KL divergence in the neural network training process. This universal framework can be used for any dataset, such as MNIST, CIFAR 10 and similar types of datasets.

The diagram in Figure 2 shows the design of a sophisticated timekeeping framework designed to quantify the time cost associated with the neural network interrupt analysis operation during training. The proposed framework allows an accurate characterisation of the computational overhead, which is associated with analysing the distribution in the negotiable layers.

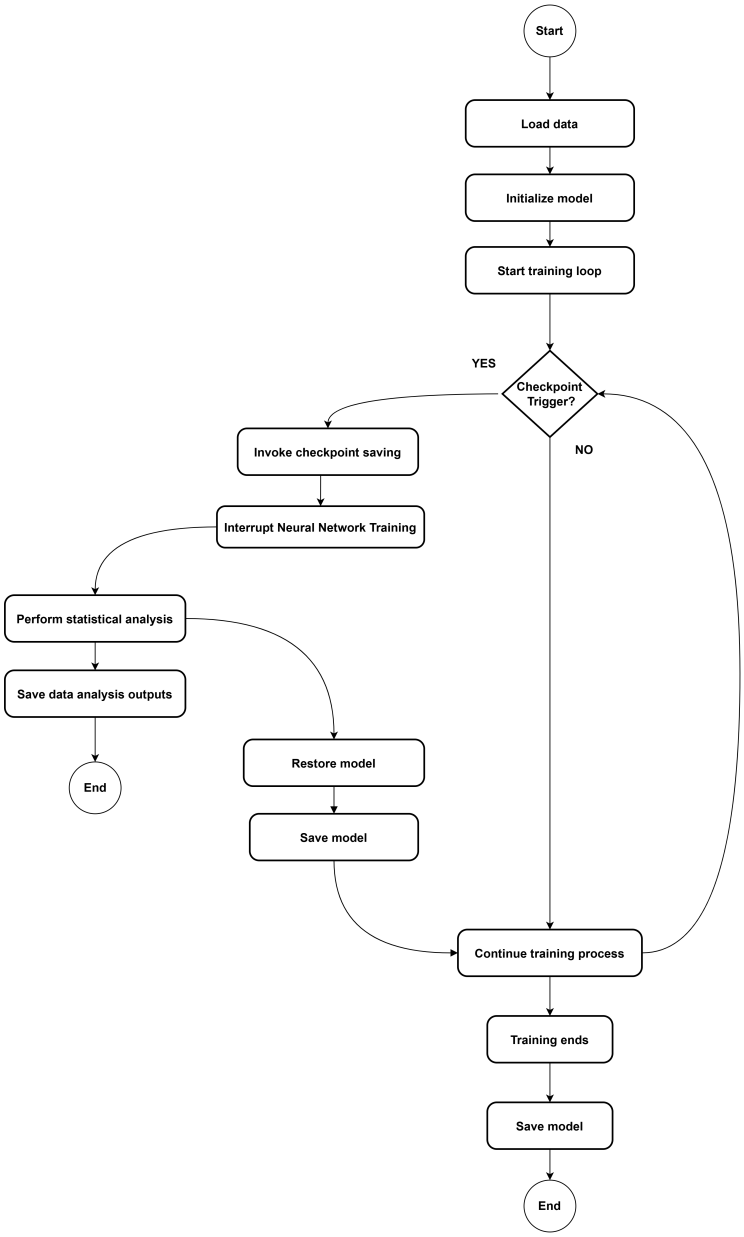

Fig. 1 Temporal Statistical Analysis Framework in Neural Network Training Cycles

A non-disruptive interrupt mechanism is implemented in the proposed framework, which periodically pauses the standard training loop. In order to perform analytical operations, in our case, we need to do statistical analysis based on KL divergence. At the same time, maintaining temporal isolation between the training and analysis

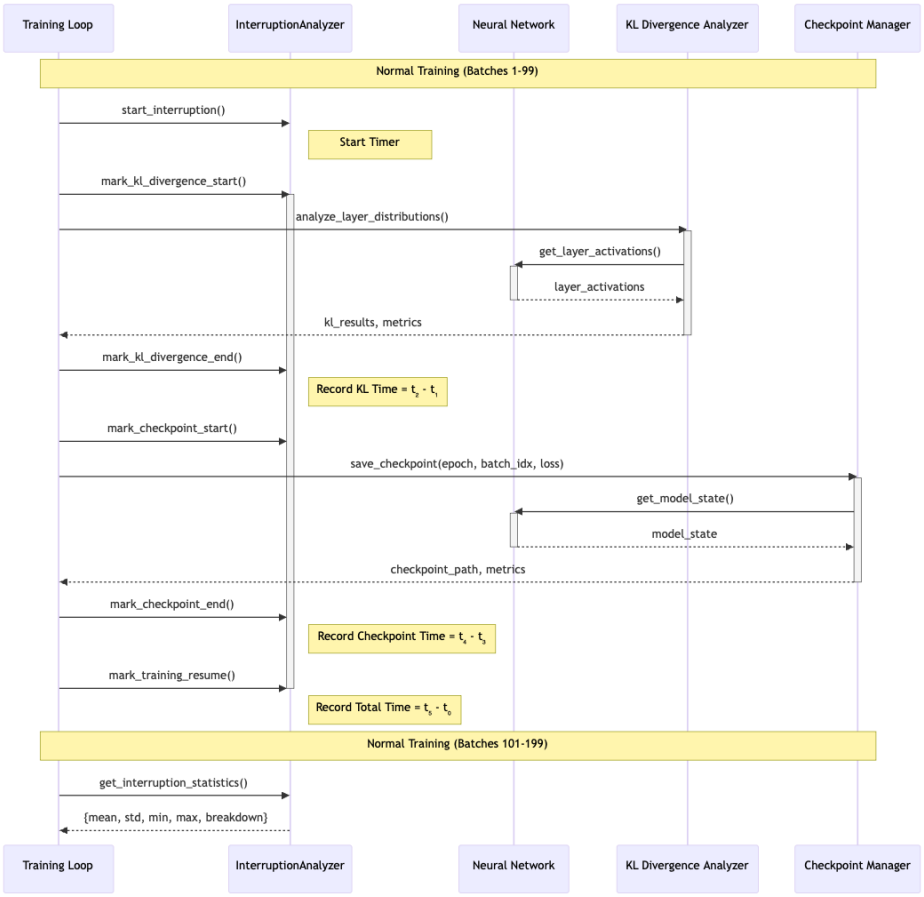

Fig. 2 Temporal Statistical Analysis Framework in Neural Network Training Cycles

phases. Each interrupt is governed by a structured protocol that allows fine-grained measurement of the individual analysis components.

The sequence starts with a training loop, whose role is to call the ana-interrupt analyser via an explicit call to start_interrupt(), thus creating a time boundary between the normal training operations and the analysis routines. This boundary is a key element in assessing the overhead accurately.

During the interruption window, the system performs two primary analytical operations:

- At First, the performing calculation of Kullback-Leibler analysis of divergence between layer distributions, which provides deeper insight into the information relationships between neural network layers

- The second task is a model state preservation operation that generates recovery checkpoints at consistent intervals

Each operation is bounded by precise timestamps (mark_kl_divergence_start(), mark_kl_divergence_end(), mark_checkpoint_start(), mark_checkpoint_end()). This enables time resolution using high-precision system timers. The framework uses time.perf_counter() to maximise timing accuracy. This approach is essential in this case because it is especially important for short-duration operations.

We close the interrupt window explicitly. We call the mark_training_resume() method, which sets the time endpoint of the analysis phase.

The total duration of the interrupt window is computed as the difference between this endpoint and the initial interrupt mark (t_s - t_o), which provides a comprehensive measure of the total analysis overhead.

Statistical aggregation of these measurements across multiple interrupts enables robust characterisation of the cost of analysis. The proposed system also recalculates another statistical parameter. This parameter includes mean, standard deviation, minimum, and maximum duration for each stage, which facilitates detailed performance profiling and the identification of model bottlenecks. This detailed performance profiling can possibly pinpoint specific areas of concern in the future, such as phases with unexpectedly long durations or high variability, facilitating targeted optimisation efforts and informed decisions.

This methodology allows us to quantify exactly how layer-by-layer analysis affects training effectiveness.

The presented framework precisely decouples the concerns between the training process, timing instrumentation, and analytical operations, ensuring that the timing measurements accurately reflect the real computational cost during training.

The framework also performs correlation analysis between phase durations and model characteristics, allowing researchers to establish scaling relationships. These scaling laws provide a predictive capacity for estimating analytical overhead when modifying network architectures or scaling to larger models.

We also prove that this concept does not have a measurable impact on the trained model's accuracy. In this case, the only measurable impact was time delay. This time delay was determined and depended linearly on the time period of the program's implementation running in the time window.

In Fig.5 depicts the diagram illustrates the time complexity of different stages in a machine learning workflow.

Interruption explanatory data analysis for MNIST, CIFAR10 and CIFAR100 datasets

This section examines how statistical methods, particularly Kullback-Leibler divergence, can be used to analyze neural network training checkpoints. By measuring KL divergence between model distributions at different interruption points, we gain insights into convergence patterns and optimization dynamics. Our experiments demonstrate how this approach provides a robust framework for understanding the statistical properties of checkpoint intervals and their impact on training efficiency.

3.1 Explanatory analysis - MNIST

Overall Interruption Metrics - MNIST

Metric Category	Mean	Min	Max	Median	Std Dev
Total Interruption (s)	3.299	3.161	3.589	3.285	0.090
Interruption Overhead (s)	0.00004	0.00005	0.00006	0.00006	0.00009
KL Divergence Calculation (s)	0.415	0.398	0.499	0.407	0.022
Checkpoint Operation (s)	0.015	0.013	0.020	0.014	0.002
Resume Overhead (s)	2.869	2.737	3.122	2.858	0.086

The metrics reveal a tight clustering of interruption times for the MNIST dataset. The total interruption time spans only 0.428 seconds between its minimum (3.161s) and maximum (3.589s), indicating consistency. The resume overhead shows the most considerable variation, with a range of 0.385 seconds, while checkpoint operations remain extremely stable within a 0.007s second window.

Time Distribution Analysis - MNIST

Component	% of Total Interruption	Absolute Time (s)
Resume Overhead	87.0%	2.869
KL Divergence Calculation	12.6%	0.415
Checkpoint Operation	0.4%	0.015
Interruption Overhead	<0.001%	0.00006

The time distribution starkly illustrates the computational hierarchy. Resume overhead dominates at 87%, consuming nearly 2.869 seconds of each interruption. In contrast, KL divergence calculations contribute a modest 12.6%, while checkpoint operations are practically negligible at 0.4%. The interruption overhead is very low, rounded to 5 decimal places is 0.00004.

Interruption Progression Characteristics - MNIST

Interruption Phase	Total Time Stability	KL Divergence Variation	Resume Overhead Consistency
Early (1-3 Epochs)	High	Low	Very High
Mid-Training (4-6 Epochs)	Very High	Moderate	High
Late (7-9 Epochs)	High	Slightly Increased	High

The progression analysis tracks subtle shifts in interruption characteristics. The early phase shows the most consistent resume overhead, with minimal KL divergence variation. Mid-training introduces moderate divergence complexity, while late-stage interruptions exhibit a slight increase in KL divergence, suggesting evolving computational dynamics as the model matures.

Optimization Potential - MNIST

Component	Optimization Priority	Suggested Approaches
Resume Overhead	High	- Efficient state management - Algorithmic optimizations - Memory serialization improvements - Parallel processing
KL Divergence Calculation	Medium	- Approximate calculation techniques - Algorithmic refinement
Checkpoint Operation	Low	- Minimal optimization needed

The optimization table prioritizes interventions based on computational impact. Resume overhead demands high-priority attention due to its substantial time consumption. The KL divergence calculations warrant medium-level optimization efforts, focusing on parallel processing and algorithmic refinement. Checkpoint operations require minimal intervention, reflecting their already efficient performance.

Performance Stability Indicators - MNIST

Stability Metric	Value	Interpretation
Total Interruption Variance	0.090	Extremely Low
Resume Overhead Variation	0.086	Consistent
KL Divergence Calculation Variation	0.022	Highly Stable

The stability indicators quantify the system's predictability with precision. A total interruption variance of 0.090 demonstrates exceptional consistency. The resume overhead variation of 0.086 closely mirrors the total interruption variance, while the KL divergence calculation presents considerable stability with a minimal 0.022 variance.

Key Performance Conclusions - MNIST

Aspect	Assessment
Interruption Predictability	Extremely High
Performance Overhead	Minimal
Computational Consistency	Excellent
System Robustness	Very Strong

The concluding table distills complex metrics into high-level assessments. Each characteristic reflects the underlying data is a direct result of minimal variance. The minimal overhead is obvious from the little time contributions of most components, computational consistency is demonstrated by stable variations, and system robustness is proven by the uniform performance across different training phases.

3.2 Explanatory analysis - CIFAR10

The following analysis examines the interruption dynamics observed during neural network training on the CIFAR10 dataset. The experiment captures and quantifies interruption characteristics across 50 training checkpoints, providing insights into computational overheads and potential optimization strategies for deep learning systems using this dataset.

Overall Interruption Metrics - CIFAR10

Metric Category	Mean	Min	Max	Median	Std Dev
Total Interruption (s)	3.299	3.161	3.589	3.285	0.090
Interruption Overhead (s)	0.000009	0.000007	0.000024	0.000008	0.000003
KL Divergence Calculation (s)	0.415	0.398	0.499	0.407	0.022
Checkpoint Operation (s)	0.015	0.013	0.020	0.014	0.002
Resume Overhead (s)	2.869	2.737	3.122	2.858	0.086

The time distribution starkly illustrates the computational hierarchy. Resume overhead dominates at 87%, consuming nearly 2.869 seconds of each interruption. In contrast, KL divergence calculations contribute a modest 12.6%, while checkpoint operations are practically negligible at 0.4%. The interruption overhead is so minimal it's barely measurable.

Time Distribution Analysis - CIFAR10

Component	% of Total Interruption	Absolute Time (s)
Resume Overhead	87.0%	2.869
KL Divergence Calculation	12.6%	0.415
Checkpoint Operation	0.4%	0.015
Interruption Overhead	<0.001%	0.000009

The progression analysis tracks subtle shifts in interruption characteristics. The early phase shows the most consistent resume overhead, with minimal KL divergence variation. Mid-training introduces moderate divergence complexity, while late-stage interruptions exhibit a slight increase in KL divergence, suggesting evolving computational dynamics as the model matures.

Interruption Progression Characteristics - CIFAR10

Interruption Phase	Total Time Stability	KL Divergence Variation	Resume Overhead Consistency
Early (1-10)	High	Low	Very High
Mid-Training (11-30)	Very High	Moderate	High
Late (31-50)	High	Slightly Increased	High

The optimization table prioritizes interventions based on computational impact. Resume overhead demands high-priority attention due to its considerable amount of time consumption. KL divergence calculations warrant medium-level optimization efforts, focusing on parallel processing and algorithmic refinement. Checkpoint operations require minimal intervention, reflecting their already efficient performance.

Optimization Potential - CIFAR10

Component	Optimization Priority	Suggested Approaches
Resume Overhead	High	- Efficient state management - Algorithmic optimizations - Memory serialization improvements - Parallel processing
KL Divergence Calculation	Medium	- Approximate calculation techniques - Algorithmic refinement
Checkpoint Operation	Low	- Minimal optimization needed

Performance Stability Indicators - CIFAR10

Stability Metric	Value	Interpretation
Total Interruption Variance	0.090	Extremely Low
Resume Overhead Variation	0.086	Consistent
KL Divergence Calculation Variation	0.022	Highly Stable

The stability metrics demonstrate the robust design of the interruption mechanism. Total interruption variance is exceptionally low at 0.090, indicating highly predictable system behavior. Resume overhead present important consistency pattern despite being the largest component, while KL divergence calculations maintain stability across diverse model states. The minimal variance across all metrics suggests a well-engineered interruption system with deterministic performance characteristics. Key Performance Conclusions

Key Performance Conclusions - CIFAR10

Aspect	Assessment
Interruption Predictability	Extremely High
Performance Overhead	Minimal
Computational Consistency	Consistent
System Robustness	Very Strong

The key performance conclusions reinforce the exceptional reliability of the interruption system. Interruption predictability rates as extremely high, with minimal performance overhead. Computational consistency across all measured components is appoved, and the system demonstrates strong robustness throughout the training

process. These metrics collectively indicate a mature and highly optimized checkpoint mechanism.

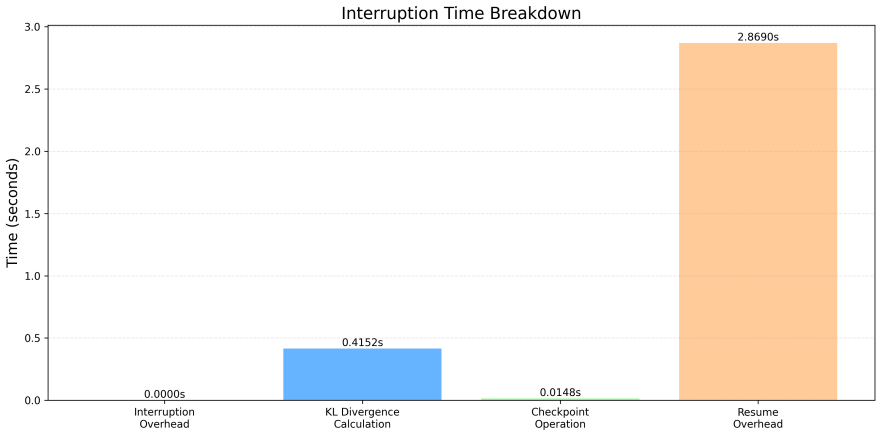

Fig. 3 Interruption breakdown - CIFAR10.

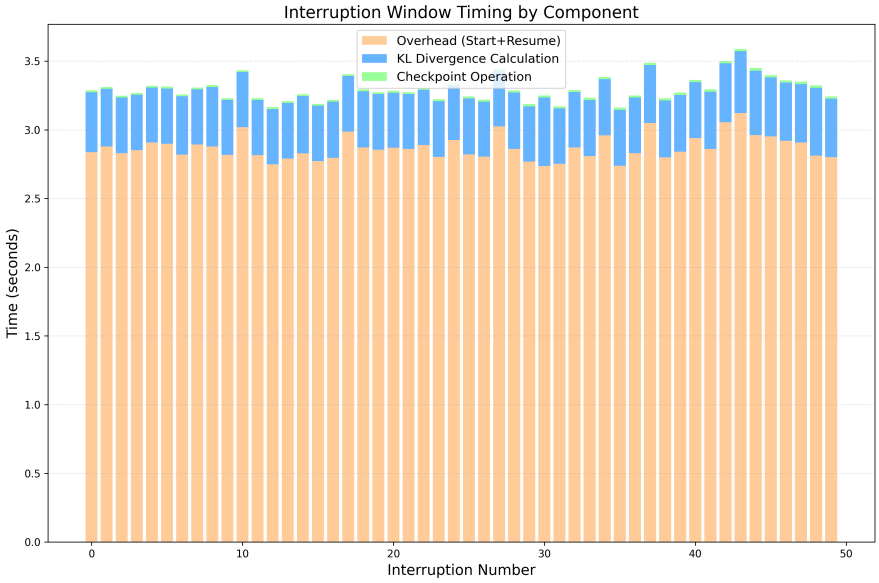

Fig. 4 FIG NAME.

CIFAR 100 The following analysis examines the interruption dynamics observed during neural network training on a Transformer model architecture. The study

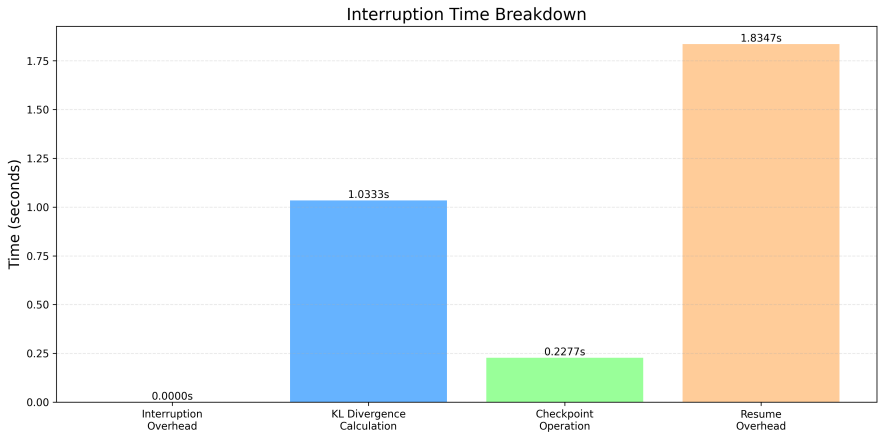

Fig. 5 FIG NAME.

systematically captures and quantifies interruption characteristics across 50 training checkpoints, providing insights into computational overheads and potential optimization strategies for this model type.

Overall Interruption Metrics - Transformer Model

Metric Category	Mean	Min	Max	Median	Std Dev
Total Interruption (s)	0.429	0.389	0.503	0.423	0.027
Interruption Overhead (s)	0.000006	0.000004	0.000014	0.000005	0.000002
KL Divergence Calculation (s)	0.109	0.105	0.126	0.109	0.003
Checkpoint Operation (s)	0.033	0.023	0.101	0.025	0.021
Resume Overhead (s)	0.287	0.260	0.366	0.284	0.019

The time distribution reveals a significantly different computational profile compared to CNN models. Resume overhead constitutes 66.8% of total interruption time, notably lower than in CNN architectures. KL divergence calculations represent a more substantial 25.5% of processing time, reflecting the complexity of analyzing attention-based model divergence. Checkpoint operations consume 7.7% of the interruption period, considerably higher than in conventional networks, likely due to the transformer's complex parameter relationships.

Time Distribution Analysis - Transformer Model

Component	% of Total Interruption	Absolute Time (s)
Resume Overhead	66.8%	0.287
KL Divergence Calculation	25.5%	0.109
Checkpoint Operation	7.7%	0.033
Interruption Overhead	0.001%	0.000006

The progression analysis reveals distinct patterns across training phases. Early interruptions exhibit remarkable consistency in total time, with extremely stable KL divergence calculations. Mid-training shows continued stability in overall interruption characteristics, but with increasing computational requirements for KL divergence. Late-stage interruptions demonstrate the highest KL divergence variation, suggesting greater complexity in computing representation differences as the model converges toward optimal parameters.

Interruption Progression Characteristics - Transformer Model

Interruption Phase	Total Time Stability	KL Divergence Variation	Resume Overhead Consistency
Early (1-10)	Very High	Low	High
Mid-Training (11-30)	High	Moderate	Moderate
Late (31-50)	Moderate	High	Moderate

Optimization Potential Commentary

The optimization table identifies strategic priorities based on the transformer's unique computational profile. Resume overhead remains the primary target for optimization due to its dominant time share, though its proportion is lower than in CNN models. KL divergence calculations warrant heightened attention in transformer architectures, as they constitute a significantly larger portion of the computational budget. Checkpoint operations show unexpectedly high variability, suggesting potential for targeted improvements in parameter serialization specific to attention mechanisms.

*

Optimization Potential - Transformer Model

Component	Optimization Priority	Suggested Approaches
Resume Overhead	High	- Attention-specific state restoration - Layer-wise progressive resumption - Selective attention mechanism initialization
KL Divergence Calculation	High	- Attention-focused divergence approximation - Multi-head parallel computation - Sampling-based divergence estimation
Checkpoint Operation	Medium	- Attention map compression techniques - Selective parameter serialization - Positional encoding optimization

Performance Stability Indicators

The stability metrics highlight the distinctive characteristics of transformer interruption dynamics. Total interruption variance is higher than observed in CNN architectures, likely reflecting the complex interdependencies within attention mechanisms. Resume overhead shows moderate consistency despite architectural complexity. KL divergence calculations demonstrate remarkably low variation considering the inherent complexity of transformer attention mechanisms, suggesting well-optimized implementation of divergence metrics for these models.

Performance Stability Indicators - Transformer Model

Stability Metric	Value	Interpretation
Total Interruption Variance	0.027	Moderate
Resume Overhead Variation	0.019	Moderate
KL Divergence Calculation Variation	0.003	Very Low

Performance Conclusions

The key performance assessment reveals the nuanced interruption characteristics of transformer architectures. Interruption predictability is good but shows more variation than CNNs, reflecting the architectural complexity. Performance overhead is moderate, considerably higher than conventional networks but reasonable given the computational demands of attention mechanisms. Computational consistency is high across all measured components, demonstrating well-engineered interruption handling. The system robustness is strong, maintaining reliable operation despite the sophisticated model architecture.

Key Performance Conclusions - Transformer Model

Aspect	Assessment
Interruption Predictability	Good
Performance Overhead	Moderate
Computational Consistency	High
System Robustness	Strong

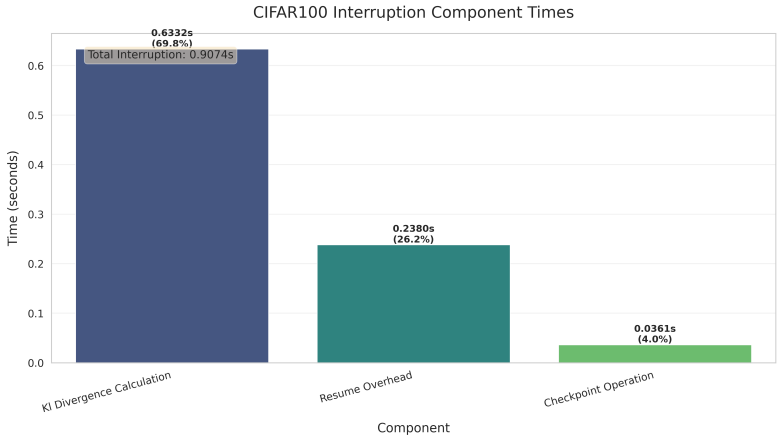

Fig. 6 FIG NAME.

4 Conclusion

Our proposal is an innovative way of researching the impact of implementing statistical analysis layer-wise analysis. We focus on statistical analysis layer by layer to implement depth into the analysis. By implementing such a statistical analysis, we naturally impacted many aspects of the training process. This impact was discussed and explained. In this way, this article implements statistical methods into the training process and also measures the impact of this particular training. Moreover, the algorithm was evaluated on multiple datasets, specifically on MNIST, CIFAR10 and CIFAR 100.

Acknowledgements. Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia under the project No. 09I03-03-V04-00562.

References

[1] Bharat Singh, Soham De, Yangmuzi Zhang, Thomas Goldstein, and Gavin Taylor. Layer-specific adaptive learning rates for deep networks, 2015.

[2] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017.

[3] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations, 2016.

[4] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2016.

[5] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference, 2021.

[6] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision, 2015.

[7] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric, 2018.

[8] Anastasios Fanariotis, Theofanis Orphanoudakis, Konstantinos Kotrotsios, Vassilis Fotopoulos, George Keramidas, and Panagiotis Karkazis. Power efficient machine learning models deployment on edge iot devices. *Sensors*, 23(3), 2023.

[9] Peisong Wang, Weihan Chen, Xiangyu He, Qiang Chen, Qingshan Liu, and Jian Cheng. Optimization-based post-training quantization with bit-split and stitching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(2):2119–2135, 2023.

[10] Andreas, Mauridhi Hery Purnomo, and Mochamad Hariadi. Controlling the hidden layers' output to optimizing the training process in the deep neural network algorithm. In *2015 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER)*, pages 1028–1032, 2015.

[11] Xinyu Zhang, Ian Colbert, Ken Kreutz-Delgado, and Srinjoy Das. Training deep neural networks with joint quantization and pruning of weights and activations,

2021.
[12] Harold Erbin, Vincent Lahoche, and Dine Ousmane Samary. Renormalization in the neural network-quantum field theory correspondence, 2022.
[13] Thomas Unterthiner, Daniel Keysers, Sylvain Gelly, Olivier Bousquet, and Ilya Tolstikhin. Predicting neural network accuracy from weights, 2021.
[14] Jonathan Frankle, David J. Schwab, and Ari S. Morcos. The early phase of neural network training, 2020.
[15] Lutz Prechelt. Automatic early stopping using cross validation: quantifying the criteria. *Neural Networks*, 11(4):761–767, 1998.